

Tài Liệu Giảng Bài Và Kịch Bản Thuyết Trình

Ứng dụng ANN làm heuristic cho A^* trong bài toán tìm đường mê cung

Nhóm Tờ Linh: Nguyễn Quốc Minh, Võ Nguyễn Tấn Tài

Dùng để luyện nói, demo, giải thích code và trả lời phản biện

Contents

1	Mục Tiêu Của Tài Liệu	2
2	Cách Mở Đầu Bài Thuyết Trình	2
3	Phần Giảng Lý Thuyết 1: Bài Toán Mê Cung Và Tìm Kiếm	3
3.1	Mê cung được biểu diễn như thế nào?	3
3.2	BFS là gì và dùng làm gì?	3
3.3	A* là gì?	3
3.4	Manhattan heuristic	4
4	Phần Giảng Lý Thuyết 2: ANN Là Gì?	4
4.1	Ý tưởng cơ bản của ANN	4
4.2	Một neuron tính gì?	4
4.3	ANN trong đề tài nhận input gì?	5
4.4	Kiến trúc ANN dùng trong đề tài	5
5	Phần Giảng Lý Thuyết 3: ANN Học Như Thế Nào?	6
5.1	Forward propagation	6
5.2	Loss và metric	6
5.3	Backpropagation là gì?	7
5.4	Epoch và batch size	7
6	Phần Quan Trọng: Validation Và Test Khác Nhau Thế Nào?	7
6.1	Ba tập dữ liệu	7
6.2	Ví dụ trực quan về validation	8
6.3	Tại sao cần tách Validation khỏi Test?	8
6.4	Quy tắc chia dữ liệu phổ biến	9
7	Các Chỉ Số Đánh Giá Cần Giải Thích	9
7.1	Accuracy là gì?	9
7.2	ROC/AUC là gì?	9
7.3	Cross-validation là gì?	10
7.4	Đọc đồ thị Loss/Accuracy qua các Epoch	10
8	Dữ Liệu Được Tạo Như Thế Nào?	11
8.1	Pipeline tạo dataset	11
8.2	Vì sao chạy BFS từ goal?	11
9	Ba Trạng Thái Thực Nghiệm Của Mô Hình	11
9.1	Underfitting	11
9.2	Overfitting	12
9.3	Good fit	12
9.4	Bảng so sánh ba thí nghiệm	13
10	Tích Hợp ANN Vào A*	13
10.1	Từ model thành heuristic	13
10.2	Demo kết quả	14
11	Kịch Bản Thuyết Trình Theo Slide	14

12 Kịch Bản Demo Trực Tiếp	15
12.1 Lệnh chạy	15
12.2 Thao tác	16
13 Giải Thích Mã Nguồn Theo Module	16
14 Câu Hỏi Phản Biện Và Trả Lời	16
15 Kết Bài	17

1 Mục Tiêu Của Tài Liệu

Tài liệu này được viết theo kiểu **giảng lại bài**, không chỉ liệt kê ý trên slide. Khi thuyết trình, người nói cần giải thích được vì sao bài toán cần ANN, ANN học như thế nào, validation khác test ra sao, và cách ANN được đưa vào A*.

Bám Sát Rubric Chấm Điểm

Tiêu chí	Cách thể hiện trong bài nói
Lý thuyết 25%	Giảng ANN từ neuron, lớp Dense, forward propagation, loss, backpropagation, epoch. Phân biệt rõ train, validation, test bằng ví dụ trực quan.
Thực nghiệm 35%	Dùng ba đồ thị <code>underfit_loss.png</code> , <code>overfit_loss.png</code> , <code>goodfit_loss.png</code> . Chỉ ra dấu hiệu underfit, overfit, good fit trên đồ thị train thật.
Sáng tạo 20%	Không chỉ train model; model ANN được tích hợp vào A* làm heuristic. Demo so sánh BFS, A* + Manhattan, A* + ANN.
Trung thực 20%	Nói rõ code tự cài theo module; AI hỗ trợ khung/debug/tổ chức nội dung; nhóm tự chạy dataset, train, đọc metric và giải thích cấu hình.

Thông điệp chính cần nhớ

Đề tài sinh map mê cung ngẫu nhiên, dùng BFS để tạo nhãn khoảng cách thật, huấn luyện ANN bằng Keras để dự đoán khoảng cách còn lại, rồi dùng dự đoán đó làm heuristic cho A*.

Ba câu phải nói chắc:

- BFS không sinh map. Map sinh ngẫu nhiên trong `maze.py`.
- BFS dùng để tạo nhãn `true_distance` cho ANN.
- ANN được xây bằng Keras API qua `tf.keras` và được đưa vào A* qua `AnnHeuristic`.

2 Cách Mở Đầu Bài Thuyết Trình

Kịch bản mở đầu 45–60 giây

Em xin chào thầy/cô và các bạn. Nhóm em thực hiện đề tài ứng dụng mạng nơ-ron nhân tạo làm hàm heuristic cho thuật toán A* trong bài toán tìm đường mê cung. Bình thường A* dùng heuristic Manhattan, tức một công thức cố định dựa trên tọa độ. Nhóm em thử thay heuristic đó bằng một mô hình ANN học từ dữ liệu. Dữ liệu được sinh từ nhiều map mê cung ngẫu nhiên, còn nhãn khoảng cách thật được tính bằng BFS. Sau khi train, mô hình ANN dự đoán chi phí còn lại đến goal và giá trị đó được đưa vào A* để quyết định thứ tự duyệt node.

3 Phần Giảng Lý Thuyết 1: Bài Toán Mê Cung Và Tìm Kiếm

3.1 Mê cung được biểu diễn như thế nào?

Mê cung là một ma trận 20x20. Mỗi ô có một trong hai trạng thái:

Giá trị	Ý nghĩa
0	Đường đi, agent có thể đứng hoặc đi qua.
1	Tường/vật cản, agent không được đi qua.

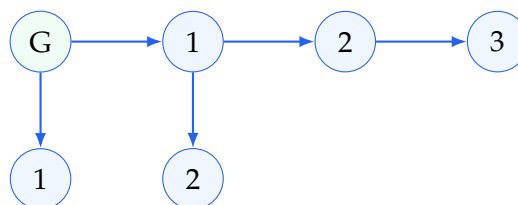
Agent đi 4 hướng: lên, xuống, trái, phải. Mỗi bước có chi phí bằng 1. Vì mọi cạnh đều cùng chi phí, bài toán này là đồ thị không trọng số.

Liên hệ mã nguồn

```
src/maze.py định nghĩa FREE = 0, WALL = 1, danh sách 4 hướng và hàm
iter_neighbors. Hàm generate_maze sinh map ngẫu nhiên bằng điều kiện
rng.random < wall_probability. Với cấu hình hiện tại, wall_probability =
0.25.
```

3.2 BFS là gì và dùng làm gì?

BFS duyệt đồ thị theo từng lớp khoảng cách. Từ một ô ban đầu, BFS duyệt tất cả ô cách 1 bước, rồi tất cả ô cách 2 bước, rồi 3 bước, v.v. Vì duyệt theo lớp như vậy, trong đồ thị không trọng số BFS cho khoảng cách ngắn nhất.



Trong đề tài, BFS có hai vai trò:

1. **Tạo nhãn học:** chạy BFS từ goal để tính `true_distance` cho mọi ô reachable.
2. **Làm thuật toán mốc:** dùng BFS để so sánh với A* + Manhattan và A* + ANN.

Điểm hay bị hỏi

BFS không dùng để sinh map. Map được sinh ngẫu nhiên trước. Sau đó BFS mới chạy trên map đã có để tính nhãn khoảng cách thật.

3.3 A* là gì?

A* là thuật toán tìm kiếm có định hướng. Nó đánh giá mỗi node bằng:

$$f(n) = g(n) + h(n)$$

Trong đó:

- $g(n)$: chi phí đã đi từ start đến node n .
- $h(n)$: chi phí ước lượng từ node n đến goal.
- $f(n)$: tổng chi phí dự đoán nếu đi qua node n .

Giải thích trực quan

Nếu BFS giống như tìm kiếm lan đều ra mọi hướng, thì A* giống như người đang cầm bản đồ và ưu tiên đi về phía có vẻ gần đích hơn. Phần “có vẻ gần đích hơn” chính là heuristic $h(n)$.

3.4 Manhattan heuristic

Trong lưới 4 hướng, heuristic Manhattan được tính bằng:

$$h(n) = |x - x_g| + |y - y_g|$$

Ví dụ: nếu agent đang ở $(x, y) = (1, 17)$, goal ở $(7, 7)$, thì:

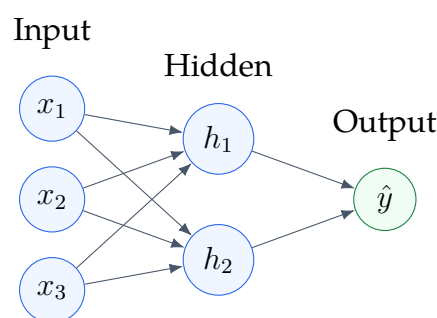
$$h(n) = |1 - 7| + |17 - 7| = 6 + 10 = 16$$

Manhattan rất nhanh nhưng chỉ dùng tọa độ. Nó không học từ dữ liệu và không trực tiếp thấy cấu trúc tường trong mê cung.

4 Phần Giảng Lý Thuyết 2: ANN Là Gì?

4.1 Ý tưởng cơ bản của ANN

ANN (Artificial Neural Network) là mô hình học máy gồm nhiều lớp tính toán. Nó nhận input, biến đổi qua các lớp ẩn, rồi tạo output. Trong đề tài này, ANN nhận trạng thái mê cung và dự đoán số bước còn lại đến goal.



4.2 Một neuron tính gì?

Một neuron nhận nhiều input, nhân với trọng số, cộng bias, rồi đưa qua hàm kích hoạt:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b = w^T x + b$$

$$a = \phi(z)$$

Trong đó ϕ là hàm kích hoạt. Trong đề tài dùng ReLU ở các lớp ẩn:

$$\text{ReLU}(z) = \max(0, z)$$

Cách giảng dễ hiểu

Hãy xem mỗi trọng số w như một nút chỉnh. Nếu một đặc trưng quan trọng, trọng số của nó có thể được học lớn hơn. Nếu đặc trưng ít quan trọng, trọng số có thể nhỏ hơn. Train ANN là quá trình tự động chỉnh các nút này để dự đoán gần nhất thật hơn.

4.3 ANN trong đề tài nhận input gì?

Mỗi trạng thái trong mê cung được đổi thành vector 10 chiều:

Nhóm feature	Ý nghĩa
$\mathbf{x}, \mathbf{y}$	Tọa độ hiện tại của agent, đã chuẩn hóa theo kích thước map.
$\text{goal_x}, \text{goal_y}$	Tọa độ goal, đã chuẩn hóa.
dx, dy	Khoảng cách tuyệt đối theo hai trục, gần giống thông tin Manhattan.
$\text{up_wall}, \text{down_wall}, \text{left_wall}, \text{right_wall}$	Bốn tín hiệu cho biết ô lân cận có phải tường/ngoài biên hay không.

Output của ANN là:

$$\hat{y} = \text{dự đoán số bước còn lại đến goal}$$

Nhân thật là:

$$y = \text{true_distance} = \text{số bước ngắn nhất do BFS tính}$$

Ví dụ một trạng thái cụ thể

Trong demo đã lưu, start là hàng 17, cột 1; goal là hàng 7, cột 7. Theo code, x là cột và y là hàng:

$$x = 1, \quad y = 17, \quad \text{goal}_x = 7, \quad \text{goal}_y = 7$$

Vì map 20x20 nên chia cho 19 để chuẩn hóa:

$$x/19 \approx 0.053, \quad y/19 \approx 0.895$$

Khoảng cách theo trục:

$$dx = |1 - 7| = 6, \quad dy = |17 - 7| = 10$$

4.4 Kiến trúc ANN dùng trong đề tài

Mô hình good fit là mô hình chính:



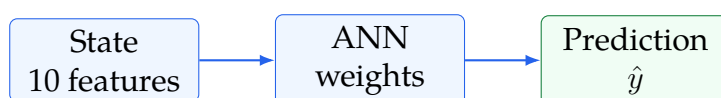
Giải thích Dropout

Dropout tạm thời bỏ ngẫu nhiên một phần neuron trong lúc train. Mục tiêu là tránh việc mô hình phụ thuộc quá mạnh vào một vài neuron, giúp giảm overfitting.

5 Phần Giảng Lý Thuyết 3: ANN Học Như Thế Nào?

5.1 Forward propagation

Forward propagation là bước đưa dữ liệu từ input qua mạng để tạo dự đoán.



Ví dụ: một ô có khoảng cách thật là 20 bước. Model dự đoán 17.5 bước. Khi đó sai số tuyệt đối của mẫu này là 2.5 bước.

5.2 Loss và metric

Vì đây là bài toán hồi quy, nhóm dùng MSE làm loss và MAE làm metric.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Ví dụ tính MAE/MSE bằng tay

Giả sử có 3 mẫu:

$$y = [20, 10, 15], \quad \hat{y} = [18, 13, 14]$$

Sai số tuyệt đối:

$$|y - \hat{y}| = [2, 3, 1]$$

$$MAE = \frac{2 + 3 + 1}{3} = 2$$

Sai số bình phương:

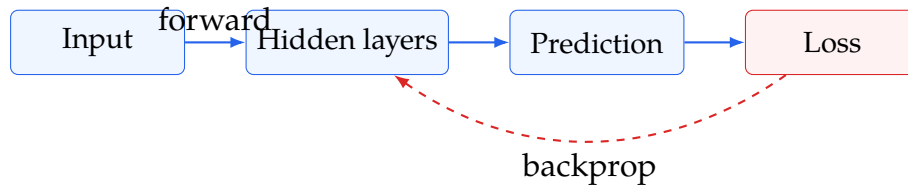
$$(y - \hat{y})^2 = [4, 9, 1]$$

$$MSE = \frac{4 + 9 + 1}{3} = 4.67$$

MAE dễ giải thích vì đơn vị là số bước. MSE phạt lỗi lớn mạnh hơn nên phù hợp làm loss khi train.

5.3 Backpropagation là gì?

Sau khi tính loss, mô hình cần biết trọng số nào làm dự đoán sai nhiều. Backpropagation truyền sai số từ output ngược về các lớp trước để tính gradient. Sau đó optimizer Adam cập nhật trọng số.



Cách nói dễ hiểu

Forward là mô hình trả lời: “tôi đoán còn 17 bước”. Loss cho biết câu trả lời đó sai bao nhiêu so với BFS. Backpropagation là quá trình rút kinh nghiệm: trọng số nào cần tăng, trọng số nào cần giảm để lần sau dự đoán gần đúng hơn.

5.4 Epoch và batch size

Một epoch là một lần mô hình học qua toàn bộ tập train. Batch size là số mẫu được đưa vào model trong một lần cập nhật. Trong code:

- Underfit: 5 epoch, batch size 64.
- Overfit: 150 epoch, batch size 32, chỉ dùng 3.000 dòng train.
- Good fit: tối đa 100 epoch, batch size 64, EarlyStopping nên dừng ở 20 epoch.

Giải thích epoch bằng ví dụ lớp học

Nếu tập train giống như một quyển bài tập có 70.000 câu, thì một epoch nghĩa là mô hình đã xem qua toàn bộ 70.000 câu đó một lần. Nếu train 5 epoch, mô hình xem lại quyển bài tập 5 lượt. Xem quá ít có thể chưa hiểu bài; xem quá nhiều với một mô hình quá lớn có thể dẫn đến học vẹt.



6 Phân Quan Trọng: Validation Và Test Khác Nhau Thế Nào?

6.1 Ba tập dữ liệu

Dataset có 100.000 dòng và được chia 70/15/15:

Train	Validation	Test
70.000 dòng	15.000	15.000

Ví dụ trực quan: học tập, kiểm tra thử, thi cuối kỳ

Có thể hiểu ba tập dữ liệu giống quá trình học của sinh viên:

- **Training set:** bài tập để học. Làm sai thì sửa ngay, tương đương model cập nhật trọng số.
- **Validation set:** bài kiểm tra thử trong lúc ôn. Không dùng để học trực tiếp từng đáp án, nhưng dùng để biết cách học hiện tại có ổn không và có cần đổi cấu hình không.
- **Testing set:** bài thi cuối kỳ. Chỉ làm một lần sau khi đã học và chọn xong phương pháp. Dùng để báo cáo năng lực cuối cùng.

Tập dữ liệu	Dùng để làm gì?
Train	Cập nhật trọng số của ANN qua backpropagation. Đây là dữ liệu mô hình thật sự học.
Validation	Không cập nhật trọng số. Dùng để theo dõi mô hình trong quá trình train, chọn cấu hình, dùng cho EarlyStopping.
Test	Không dùng trong lúc chọn mô hình. Chỉ dùng sau cùng để đánh giá kết quả cuối cùng một cách khách quan.

6.2 Ví dụ trực quan về validation

Giả sử ta đang thử ba mô hình:

- Mô hình A quá nhỏ: validation loss cao.
- Mô hình B quá lớn: train loss thấp nhưng validation loss tăng.
- Mô hình C vừa phải: validation loss giảm và ổn định.

Ta dùng validation để chọn mô hình C. Sau khi đã chọn xong, mới dùng test để báo cáo kết quả cuối cùng.

6.3 Tại sao cần tách Validation khỏi Test?

Nếu dùng cùng một tập dữ liệu vừa để chỉnh mô hình vừa để báo cáo điểm cuối cùng, kết quả sẽ không còn khách quan. Khi ta nhìn validation loss để quyết định đổi kiến trúc, thêm Dropout, dùng EarlyStopping hoặc đổi số epoch, ta đang gián tiếp tối ưu theo validation. Vì vậy cần một tập test riêng để “thi cuối kỳ” sau khi mọi quyết định đã xong.

Hành động	Dùng tập nào?
Cập nhật trọng số qua backpropagation	Training set
Chọn kiến trúc, theo dõi overfit, EarlyStopping	Validation set
Báo cáo kết quả cuối cùng trong bảng metrics	Testing set

6.4 Quy tắc chia dữ liệu phổ biến

Không có một tỉ lệ duy nhất đúng cho mọi bài toán, nhưng thường gặp:

- **70/15/15:** 70% train, 15% validation, 15% test. Đề tài này dùng tỉ lệ này.
- **80/10/10:** dùng khi muốn dành nhiều dữ liệu hơn cho train.
- Nếu dữ liệu rất ít, có thể dùng cross-validation để đánh giá ổn định hơn.

Điểm phản biện hay gặp

Nếu dùng test nhiều lần để chọn cấu hình, test không còn khách quan. Khi đó mô hình đã “nhìn gián tiếp” vào test. Vì vậy trong báo cáo, test chỉ dùng để báo cáo cuối cùng: underfit MAE 2.669, overfit MAE 3.064, good fit MAE 2.284.

7 Các Chỉ Số Đánh Giá Cần Giải Thích

7.1 Accuracy là gì?

Accuracy là độ chính xác trong bài toán phân loại. Công thức:

$$Accuracy = \frac{\text{số mẫu dự đoán đúng}}{\text{tổng số mẫu}}$$

Ví dụ phân loại ảnh mèo/chó: nếu có 100 ảnh và model đoán đúng 92 ảnh thì Accuracy = 92%.

Liên hệ với đề tài

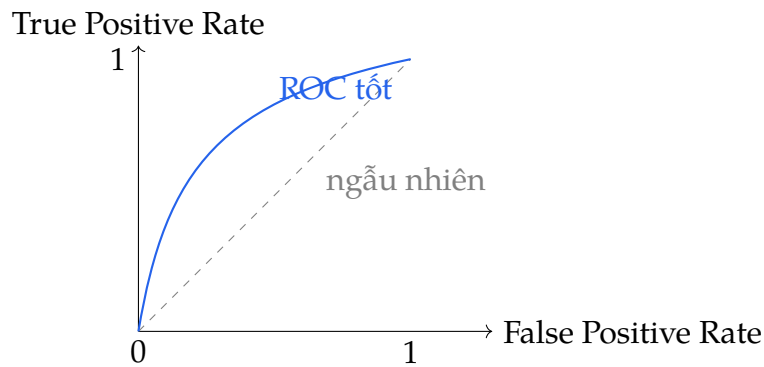
Đề tài này không dùng Accuracy làm chỉ số chính vì ANN không phân loại “đúng/sai”. ANN dự đoán một số thực là khoảng cách còn lại đến goal. Vì vậy đây là bài toán hồi quy, dùng MSE và MAE phù hợp hơn.

7.2 ROC/AUC là gì?

ROC/AUC thường dùng cho bài toán phân loại nhị phân, ví dụ phân biệt email spam/không spam. Model thường trả về một xác suất. Khi thay đổi ngưỡng quyết định, ta được nhiều cặp:

- **True Positive Rate:** tỉ lệ mẫu dương được phát hiện đúng.
- **False Positive Rate:** tỉ lệ mẫu âm bị báo nhầm là dương.

ROC curve vẽ True Positive Rate theo False Positive Rate. AUC là diện tích dưới đường ROC. AUC càng gần 1 thì khả năng phân biệt hai lớp càng tốt.



Liên hệ với đề tài

ROC/AUC không phải chỉ số chính của đề tài vì output không phải xác suất phân loại. Tuy nhiên nhóm cần hiểu khái niệm để phân biệt: classification dùng Accuracy/ROC/AUC; regression dùng MSE/MAE.

7.3 Cross-validation là gì?

Cross-validation là kỹ thuật kiểm tra chéo dữ liệu. Dữ liệu được chia thành nhiều phần (fold). Mỗi lần chọn một fold làm validation, các fold còn lại làm train. Sau nhiều lần, lấy trung bình kết quả.

Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	
Val	Train	Train	Train	Train	Lần 1
Train	Val	Train	Train	Train	Lần 2

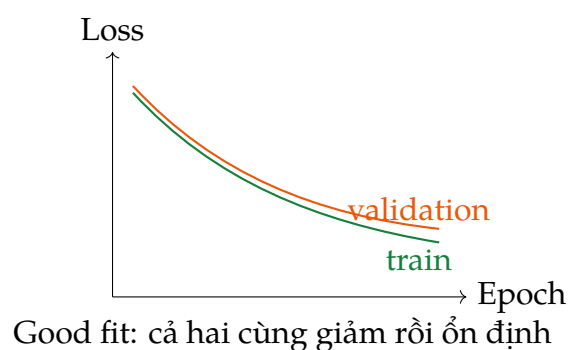
Trong đề tài, 5-fold cross-validation của mô hình good fit cho kết quả trung bình:

$$\text{Mean MAE} = 2.223, \quad \text{Mean MSE} = 14.859$$

Ý nghĩa: kết quả không chỉ tốt ở một lần chia dữ liệu duy nhất, mà tương đối ổn định qua nhiều cách chia.

7.4 Đọc đồ thị Loss/Accuracy qua các Epoch

Với bài toán phân loại, ta thường nhìn loss và accuracy theo epoch. Với bài toán hồi quy như đề tài này, ta nhìn loss và MAE theo epoch. Cách đọc vẫn giống nhau: so sánh đường train và validation.



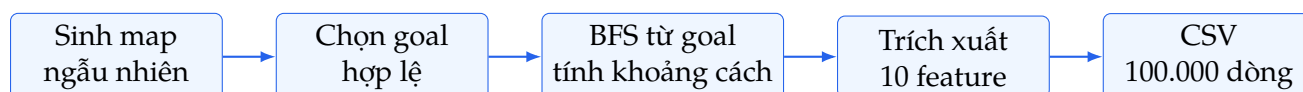
Dạng đồ thị	Cách hiểu
Train loss và validation loss đều cao	Underfitting: mô hình chưa học đủ.
Train loss giảm, validation loss tăng/xấu	Overfitting: mô hình học quá kỹ train.
Train loss và validation loss cùng giảm rồi ổn định	Good fit: mô hình học tương đối tốt và tổng quát hơn.

Câu nói khi trình bày phần chỉ số

Trong bài này, em vẫn giải thích Accuracy và ROC/AUC vì đây là các chỉ số học máy phổ biến. Tuy nhiên vì mô hình của nhóm dự đoán số bước còn lại, không phân loại nhãn, nên chỉ số chính của đề tài là MSE và MAE. Đồ thị nhóm xuất ra là loss và MAE theo epoch, tương đương cách đọc loss/accuracy trong bài toán phân loại.

8 Dữ Liệu Được Tạo Như Thế Nào?

8.1 Pipeline tạo dataset



Liên hệ code

- `dataset.py`: sinh từng map, chọn goal, chạy `bfs_distances_to_goal`.
- `features.py`: đổi mỗi ô reachable thành vector 10 feature.
- `generate_dataset.py`: tạo file `data/maze_dataset.csv`.

8.2 Vì sao chạy BFS từ goal?

Nếu chạy BFS riêng cho từng ô, mỗi ô phải chạy một lần, rất tốn thời gian. Thay vào đó, chạy BFS một lần từ goal sẽ thu được khoảng cách ngắn nhất từ mọi ô reachable đến goal.

Cách nói trên lớp

Ta có thể xem goal như trung tâm phát sóng. BFS lan từ goal ra ngoài; ô nào được chạm ở lớp thứ 5 thì cách goal 5 bước, ô nào ở lớp thứ 20 thì cách goal 20 bước. Như vậy một lần BFS tạo được nhãn cho nhiều trạng thái.

9 Ba Trạng Thái Thực Nghiệm Của Mô Hình

9.1 Underfitting

Underfitting xảy ra khi mô hình quá đơn giản hoặc học quá ít, nên không bắt được quy luật dữ liệu.

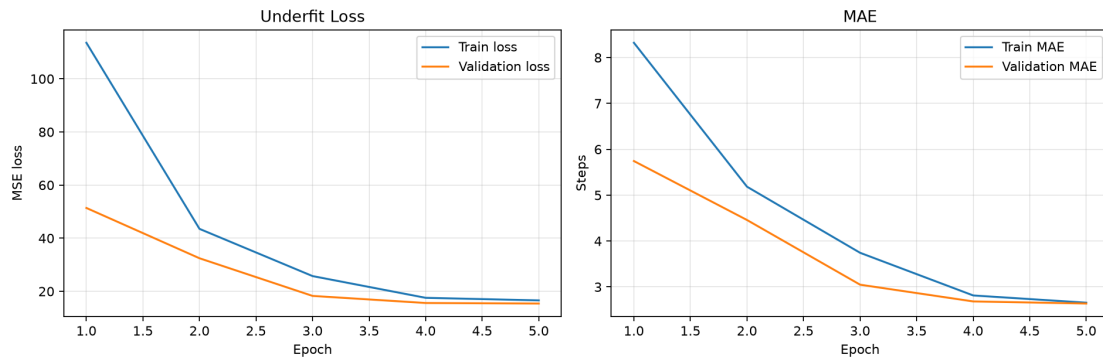


Figure 1: Đồ thị huấn luyện thật của mô hình underfit

Cách trình bày hình underfit

Ở thí nghiệm underfit, mạng chỉ có Dense(4) và output, train 5 epoch. Mô hình quá nhỏ nên sai số test còn cao: Test MSE = 16.322, Test MAE = 2.669. Khi nói, chỉ vào đồ thị loss và MAE để nhấn mạnh rằng mô hình chưa học đủ tốt.

9.2 Overfitting

Overfitting xảy ra khi mô hình học quá kỹ tập train nhưng tổng quát kém trên validation/test. Trong đề tài, overfit được tạo bằng mạng lớn và chỉ train trên 3.000 dòng.

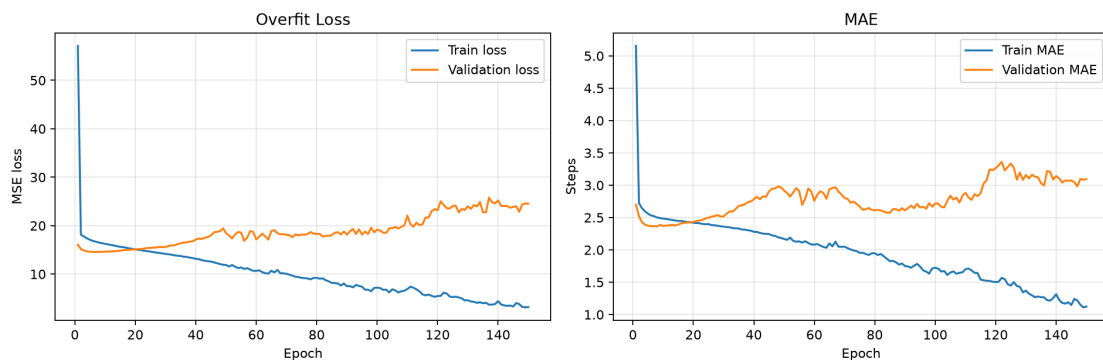


Figure 2: Đồ thị huấn luyện thật của mô hình overfit

Cách trình bày hình overfit

Mạng overfit có Dense(512), Dense(512), Dense(256), Dense(128), train 150 epoch nhưng chỉ dùng 3.000 dòng train. Dấu hiệu cần chỉ ra là train loss có thể giảm nhưng validation loss không tốt tương ứng. Test MAE = 3.064, kém hơn good fit.

9.3 Good fit

Good fit là trạng thái mô hình học đủ tốt trên train và vẫn tổng quát tốt trên validation/test. Mô hình chính dùng Dense(64), Dropout(0.2), Dense(32), Dense(16), output.

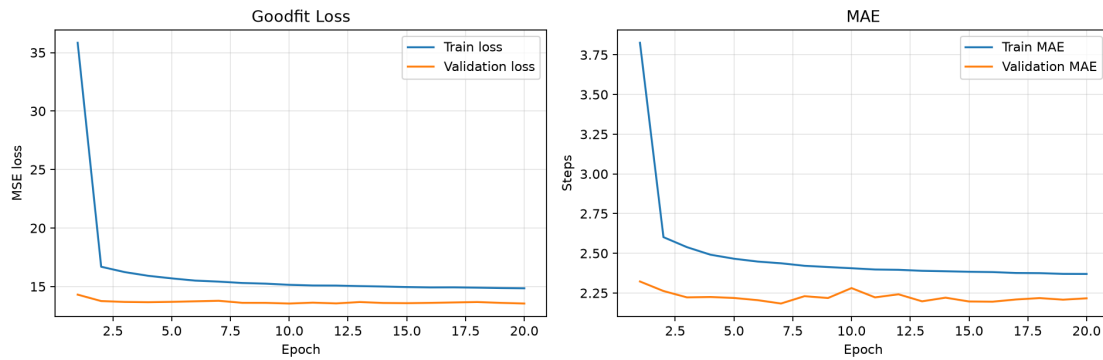


Figure 3: Đồ thị huấn luyện thật của mô hình good fit

Cách trình bày hình good fit

Mô hình good fit có kiến trúc vừa phải, dùng Dropout và EarlyStopping. Cấu hình tối đa 100 epoch nhưng dừng ở 20 epoch vì validation loss không cải thiện thêm. Kết quả tốt nhất: Test MSE = 14.300, Test MAE = 2.284.

9.4 Bảng so sánh ba thí nghiệm

Mô hình	Train rows	Epoch	Test MSE	Test MAE
Underfit	70.000	5	16.322	2.669
Overfit	3.000	150	24.811	3.064
Good fit	70.000	20	14.300	2.284

10 Tích Hợp ANN Vào A*

10.1 Từ model thành heuristic

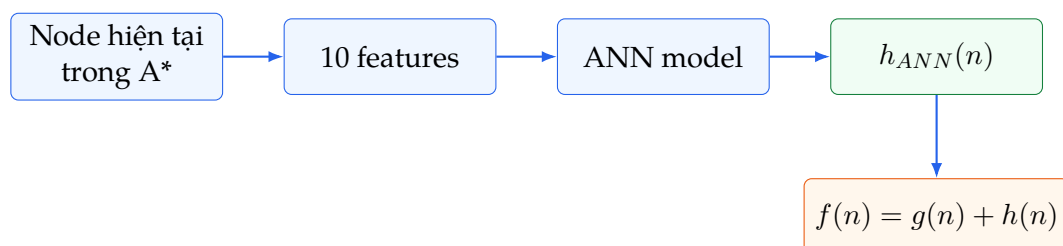
Sau khi train, model good fit được lưu thành:

- `models/ann_heuristic_goodfit.keras`: model Keras.
- `models/ann_heuristic_goodfit.tflite`: bản TensorFlow Lite để suy luận trong demo.

Trong A*, chỉ cần một hàm heuristic có dạng:

```
heuristic(position, goal) -> float
```

AnnHeuristic trong `heuristics.py` làm đúng việc này: nhận `position`, `goal`, gọi `state_to_features`, đưa vào TFLite model và trả về dự đoán khoảng cách.



10.2 Demo kết quả

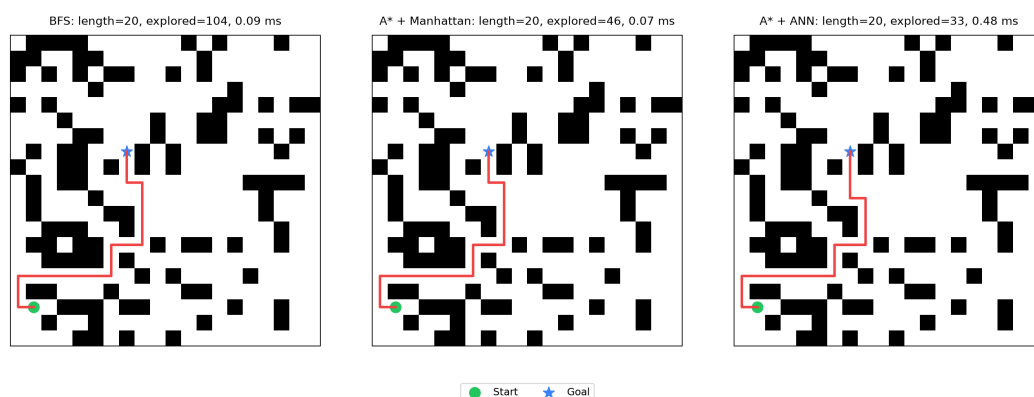


Figure 4: Ảnh demo so sánh BFS, A* + Manhattan và A* + ANN

Thuật toán	Độ dài path	Node duyệt	Thời gian
BFS	20	104	0.09 ms
A* + Manhattan	20	46	0.07 ms
A* + ANN	20	33	0.48 ms

Cách nói khi demo

Trong demo này, cả ba thuật toán đều tìm được đường dài 20 bước. BFS duyệt nhiều nhất vì không có heuristic. A* + Manhattan giảm node duyệt nhờ định hướng theo tọa độ. A* + ANN duyệt 33 node, ít nhất trong demo, vì heuristic học từ dữ liệu tạo thứ tự ưu tiên khác Manhattan. Tuy nhiên thời gian ANN cao hơn Manhattan do phải gọi model suy luận.

11 Kịch Bản Thuyết Trình Theo Slide

Slide 1: Tên đề tài

Nói

Nhóm em trình bày đề tài ứng dụng ANN làm heuristic cho A*. Thay vì chỉ dùng công thức Manhattan cố định, nhóm em huấn luyện một model để dự đoán chi phí còn lại đến goal.

Slide 2: Mục tiêu

Nói

Mục tiêu gồm ba phần: hiểu ANN học như thế nào, chứng minh ba trạng thái underfit-overfit-goodfit bằng đồ thị train thật, và tích hợp ANN vào A* để demo so sánh.

Slide 3–6: BFS, A*, Manhattan

Nói

Trước khi vào ANN, em giải thích thuật toán nền. BFS đảm bảo khoảng cách ngắn nhất trong lưới không trọng số nhưng duyệt nhiều. A* dùng $f(n) = g(n) + h(n)$. Manhattan là heuristic đơn giản dựa trên tọa độ, nhanh nhưng không học từ dữ liệu.

Slide 7–11: ANN và dữ liệu

Nói

ANN nhận 10 feature của trạng thái, gồm tọa độ, khoảng cách tương đối và tường lân cận. Output là khoảng cách còn lại. Nhân khoảng cách được BFS tính. Đây là bài toán hồi quy nên dùng MSE/MAE, không dùng Accuracy.

Slide 12–14: Underfit, overfit, good fit

Nói

Ba slide này là phần thực nghiệm quan trọng nhất. Em sẽ chỉ vào từng đồ thị: underfit do mạng quá nhỏ, overfit do mạng quá lớn và train lâu trên ít dữ liệu, good fit do cấu hình vừa phải có Dropout và EarlyStopping. Mô hình good fit là mô hình dùng cho demo A* + ANN.

Slide 15–16: Cross-validation và demo

Nói

Cross-validation kiểm tra kết quả có ổn định không. Demo cho thấy khi tích hợp ANN vào A*, số node duyệt trong map minh họa giảm còn 33, thấp hơn BFS và A* + Manhattan.

Slide 17–18: Nhận xét, hạn chế, minh bạch AI

Nói

Nhóm không khẳng định ANN luôn thắng Manhattan. Kết quả chứng minh pipeline hoạt động và có demo minh họa. AI hỗ trợ khung code/debug/tổ chức nội dung; nhóm tự chạy lại kết quả, train model và giải thích số liệu.

12 Kịch Bản Demo Trực Tiếp

12.1 Lệnh chạy

```
python -m src.ui
```

12.2 Thao tác

1. Nhấn `R` để random map mới.
2. Nhấn `A` hoặc `Run All` để chạy BFS, $A^* + \text{Manhattan}$, $A^* + \text{ANN}$.
3. Cho người xem thấy node được animate và đánh số.
4. Nhấn `0` hoặc `Show All` để hiển thị cả ba path.
5. Nếu cần chạy riêng: 1 là BFS, 2 là $A^* + \text{Manhattan}$, 3 là $A^* + \text{ANN}$.

Nếu demo live lỗi

Nếu máy không mở được Pygame hoặc TensorFlow, dùng ảnh tĩnh `outputs/demo_path.png` và bảng `outputs/demo_metrics.json`. Nói rõ đây là kết quả đã chạy trước từ cùng mã nguồn.

13 Giải Thích Mã Nguồn Theo Module

File	Vai trò cần nắm
<code>maze.py</code>	Sinh map ngẫu nhiên, định nghĩa ô tường/đường đi, duyệt 4 láng giềng.
<code>dataset.py</code>	Sinh nhiều map, chọn goal, chạy BFS để lấy khoảng cách thật.
<code>features.py</code>	Tạo 10 feature, tạo dòng dataset, tách x và y.
<code>search.py</code>	Cài BFS, A^* , Manhattan, reconstruct path, lưu thứ tự node duyệt.
<code>models.py</code>	Định nghĩa underfit, overfit, good fit bằng <code>tf.keras</code> .
<code>train.py</code>	Chia train/validation/test, train model, lưu <code>.keras</code> , export <code>.tflite</code> , vẽ loss.
<code>cross_validate.py</code>	Chạy 5-fold cross-validation.
<code>heuristics.py</code>	Bọc TFLite model thành hàm heuristic cho A^* .
<code>demo.py</code>	Chạy so sánh và lưu ảnh/metrics.
<code>ui.py</code>	UI Pygame: random map, run all, animate node, show path.

14 Câu Hỏi Phản Biện Và Trả Lời

1. ANN trong bài này là gì?

ANN là mô hình hồi quy nhận 10 feature của trạng thái mê cung và dự đoán khoảng cách còn lại đến goal. Model được xây bằng Keras qua `tf.keras.Sequential`.

2. Model học như thế nào?

Forward tạo dự đoán, MSE đo sai số, backpropagation tính gradient, Adam cập nhật trọng số. Quá trình lặp qua nhiều epoch.

3. Validation và Test khác nhau thế nào?

Validation dùng trong quá trình chọn cấu hình và EarlyStopping; test chỉ dùng cuối cùng để đánh giá khách quan. Không dùng test để điều chỉnh model.

4. Có dùng Keras không?

Có. Code dùng `tf.keras.Sequential`, `Dense`, `Dropout`, `Adam`, `MeanAbsoluteError`, `EarlyStopping`. File model lưu ở định dạng `.keras`.

5. BFS có sinh map không?

Không. Map sinh ngẫu nhiên. BFS dùng để gán nhãn `true_distance` và làm thuật toán mốc so sánh.

6. Vì sao A* + ANN chậm hơn Manhattan nhưng vẫn được dùng?

Manhattan là công thức rất nhanh. ANN phải gọi model suy luận nên chậm hơn, nhưng mục tiêu đề tài là chứng minh có thể tích hợp heuristic học từ dữ liệu vào A* và so sánh số node duyệt.

15 Kết Bài

Câu kết nên học thuộc

Qua đề tài này, nhóm em xây dựng được một pipeline hoàn chỉnh: sinh map ngẫu nhiên, dùng BFS tạo nhãn, train ANN bằng Keras, phân tích underfit-overfit-goodfit bằng đồ thị train thật, và tích hợp mô hình good fit vào A*. Điểm chính của đề tài là sự kết hợp giữa thuật toán tìm kiếm truyền thống và heuristic học từ dữ liệu.